

On Veracity Handling in Document Stores: A Novel Technique Based on L-graded Logic Running on the J-CO Framework

Maxime Deforche¹, Guy De Tré¹, Paolo Fosci², and Giuseppe Psaila²

¹ Ghent University, Dept. of Telecommunications and Information Processing,
St.-Pietersnieuwstraat 41, 9000 Ghent (Belgium),

`{Maxime.Deforche,Guy.DeTre}@UGent.be`

² University of Bergamo,

Dept. of Management, Information and Production Engineering

Viale Marconi 5, 24044 Dalmine (BG), Italy

`{paolo.fosci,giuseppe.psaila}@unibg.it`

Abstract. Data veracity denotes the trustworthiness and reliability of information and remains a primary challenge in managing big data. Managing data veracity in *NoSQL* database systems (such as *JSON*-based document stores) remains an open challenge, whereas computational intelligence techniques like fuzzy logic offer promising tools for addressing it. Specifically, a fuzzy logic framework using a specific kind of truth values, called L-grades, has been designed to assess trust in flexible query answering. In an independent work, the *J-CO* Framework has been developed to provide flexible query answering on top of *NoSQL* document stores. The present work studies the integration of both frameworks: it proposes a novel technique, based on L-grade logic and demonstrates how flexible querying of geo-tagged documents with missing and imprecise data, referring to a geographical hierarchy, can benefit from data veracity handling. The technique is illustrated by a practical implementation as a script executed by the *J-CO* Framework that queries the Database of historic Byzantine Book Epigrams (DBBE). An experimental evaluation demonstrates the potential of the approach.

Keywords: Big Data · *NoSQL* · veracity handling · flexible query answering · L-grade fuzzy sets · *J-CO* Framework.

1 Introduction

Research in flexible query answering primarily aims to align data retrieval with needs and search intentions of users in data sources often managed by database management systems. Fuzzy logic and its underlying fuzzy set theory have traditionally been intensively used in flexible query-answering approaches [1].

Over the past decades, the so-called “big data (management) challenges” [4] have emerged: the “Volume” of data has grown impressively; the “Variety” of datasets has become increasingly wider; requirements about “Velocity” (execution speed) are very demanding; full confidence in the query results (“Veracity”)

is no longer guaranteed. Conventional database management systems fail to efficiently handle big data without causing information loss. Consequently, “Not only SQL” (NoSQL) systems have been widely adopted to address this problem [12]. Current NoSQL solutions address Volume, Variety, and Velocity; however, the price to pay is often renouncing the native maintenance of data consistency and to advanced facilities for manipulating and querying data. Indeed, each *NoSQL* approach (key-value store, document store, column store, and graph database) offers a specific trade-off between solving big-data challenges and providing advanced query facilities. However, handling veracity of big data is still subject to scientific research and is not yet at the stage of generic, yet scalable solutions.

Data veracity is a complex, context-dependent, and pervasive phenomenon. Moreover, a lack of trust propagates from source data through query processing to the final outputs, impacting the reliability of these outputs [3]. This is especially true when large, diverse data sources of different, sometimes unknown origins are involved. Adequately informing users on the confidence they can have in query results is therefore crucial. This issue requires techniques to assess the (known or estimated) veracity of used data, so as to quantify the veracity of each element in a query result. In [5], “L-grade” logic has been proposed as an enrichment to (graded) fuzzy logic, so as to explicitly assess veracity of results.

Recently, the *J-CO* Framework has been proposed to provide advanced query capabilities on top of a document store, such as *MongoDB*; particularly, it provides flexible query-answering facilities [11], which encompass the ability to deal with L-grade logic. Consequently, the *J-CO* Framework is the natural tested for evaluating the application of the L-grade model towards data veracity.

As data veracity can manifest in different ways and for the sake of illustration, this paper focuses on veracity issues in query results arising from missing or imprecise hierarchical geographical tagging. A technique based on L-grades is proposed to assess and handle veracity in this context. The proposed technique is implemented as a script executed by the *J-CO* Framework and applied on the Database of Byzantine Book Epigrams (DBBE), which supports studies on ancient Greek literature. This real-life application case provides a concrete proof of concept of the novel technique, showing its native ability to deal with veracity.

The remainder of this paper is organised as follows: Section 2 provides some preliminaries and related work on L-grades, veracity issues in the DBBE database, and the *J-CO* Framework. Section 3 presents the technique for geographical querying, its implementation on the basis of the *J-CO* Framework, and a preliminary evaluation. Section 4 draws the conclusions and sketches future work.

2 Preliminaries and Related Work

This section subsequently gives some preliminaries on L-grades (Sect. 2.1), on handling veracity in the DBBE database (Sect. 2.2), and on the *J-CO* Framework (Sect. 2.3).

2.1 L-grade Logic

L-grade logic has been proposed in [5] as an extension of graded (fuzzy) logic, in such a way two gradual values are paired: the first one is a gradual truth value $s \in [0, 1]$ (degree of “satisfaction”); the second one is a gradual truth value $c \in [0, 1]$ (degree of “confidence” in s). The motivation for doing so is the need for explicitly estimate trust in (results of) logic reasoning, i.e., for assessing veracity of data sources.

Definition and Semantics. An *L-grade* is a pair

$$l = (s, c), \text{ where } s, c \in [0, 1]. \quad (1)$$

An L-grade is defined to be an ordered pair of truth values s and c . The first truth value s is interpreted as a *satisfaction grade*, denoting, for example, to what extent a data value stored in a database satisfies a (fuzzy) query criterion. The second truth value c is interpreted as a *confidence grade* that reflects the confidence in s . As both s and c belong to the unit interval, ranging from 0 (no satisfaction/confidence) to 1 (full satisfaction/confidence), satisfaction and confidence are considered to be gradual, modelling degrees of truth. The set of all L-grades is denoted by $\mathbb{L} = [0, 1]^2$.

Two L-grades $l_1 = (s_1, c_1)$ and $l_2 = (s_2, c_2)$ are equal if and only if their corresponding components are equal, i.e.,

$$(s_1 = s_2) \wedge (c_1 = c_2) \Leftrightarrow l_1 = l_2. \quad (2)$$

L-grades are, by definition, special cases of Z-numbers as defined by L.A. Zadeh in [16]. Indeed, a Z-number $Z = (A, B)$ is defined by an ordered pair of fuzzy numbers A and B . Considering a linguistic variable X , A represents a fuzzy restriction $R(X)$ on the possible (numerical) values X can take, whereas B represents a measure of the reliability one has in A . L-grades follow these semantics and are moreover the simplest form of Z-numbers as the simplest form of a fuzzy number is a crisp number $x = \{(x, 1)\}$.

Logical operators for L-grades. In order to facilitate reasoning, logical operators for L-grades are required. In [5], the usual basic operators, i.e., negation, conjunction and disjunction, are proposed; furthermore, some of their mathematical properties are studied. These operators are defined as follows:

- Negation ($\neg$):

$$\forall (s, c) \in \mathbb{L} : \neg(s, c) = (N(s), c), \quad (3)$$

where N denotes the standard fuzzy negation $N : [0, 1] \Rightarrow [0, 1] : x \mapsto 1 - x$.

- Conjunction (based on a t-norm $\top$):

$$\begin{aligned} &\forall (s_1, c_1), (s_2, c_2) \in \mathbb{L} : \\ &\top((s_1, c_1), (s_2, c_2)) = \begin{cases} (\top(s_1, s_2), \max_{s_i=0} c_i) & \text{if } s_1 = 0 \text{ or } s_2 = 0, \\ (\top(s_1, s_2), \min(c_1, c_2)) & \text{if } s_1 \neq 0 \text{ and } s_2 \neq 0 \end{cases} \quad (4) \end{aligned}$$

where $\max_{s_i=0} c_i$ denotes the maximum confidence grade c_i having an associated satisfaction grade $s_i = 0$.

– Disjunction (based on the dual t-conorm $\perp$):

$$\begin{aligned} & \forall (s_1, c_1), (s_2, c_2) \in \mathbb{L} : \\ & \perp((s_1, c_1), (s_2, c_2)) = \begin{cases} (\perp(s_1, s_2), \max_{s_i=1} c_i) & \text{if } s_1 = 1 \text{ or } s_2 = 1, \\ (\perp(s_1, s_2), \min(c_1, c_2)) & \text{if } s_1 \neq 1 \text{ and } s_2 \neq 1 \end{cases} \end{aligned} \quad (5)$$

where $\max_{s_i=1} c_i$ denotes the maximum confidence grade c_i having an associated suitability grade $s_i = 1$.

Ranking L-grades. Ranking L-grades has been studied in [6] and is important for presenting results of flexible query answering. In an L-grade $l = (s, c)$, s and c are orthogonal concepts, so both can be chosen as main ranking criteria. Hereafter, Formulas report two comparison operators, based on lexicographic ordering,

$$\begin{aligned} l_i \succ_{lex}^s l_j & \Leftrightarrow (s_i > s_j) \vee (s_i = s_j \wedge c_i > c_j) \\ l_i \succ_{lex}^c l_j & \Leftrightarrow (c_i > c_j) \vee (c_i = c_j \wedge s_i > s_j) \end{aligned} \quad (6)$$

When satisfaction (resp. confidence) is preferred over confidence (resp. satisfaction), the order relation $\succ_{lex}^s$ (resp. $\succ_{lex}^c$) can be used. If required, a threshold value $0 \leq \sigma_s \leq 1$ (resp. $0 \leq \sigma_c \leq 1$) could be used to discard elements of a query result that have an insufficient L-grade, i.e., discarding elements with an L-grade $l = (s, c)$ for which $s < \sigma_s$ (resp. $c < \sigma_c$).

2.2 Veracity Issues in the DBBE (Meta-)Data

The Database of Byzantine Book Epigrams¹ (DBBE) [14] contains a collection of Byzantine poems found in the margins of medieval manuscripts, supplemented by extensive metadata including geographical and temporal attributes. However, this metadata exhibits significant veracity issues, particularly regarding geographical information, which necessitates flexible querying techniques capable of assessing result reliability. This paper demonstrates how the approach based on L-grade addresses these challenges within the context of the DBBE.

Figure 1a illustrates a simplified epigram from the DBBE, represented as a *JSON* document. In this paper, only a few fields are considered, i.e., the epigram identifier (field `epigramId`), the textual content (field `text`, represented as an array of rows, irrelevant for the content of this paper), and the geographical origin of the epigram (field `placeId`), the latter of which references a node in a geographical hierarchy, as described below.

The DBBE organises geographical information into a hierarchy tree comprising five distinct levels of granularity: **country**, **part**, **region**, **city**, and **location**. Each epigram is mapped to a specific node within this tree: deeper nodes yield fine-grained details, while nodes nearer to the root offer only coarse or partial descriptions of its geography.

¹ DBBE: <https://www.dbbe.ugent.be/>, accessed on 30/01/2026.

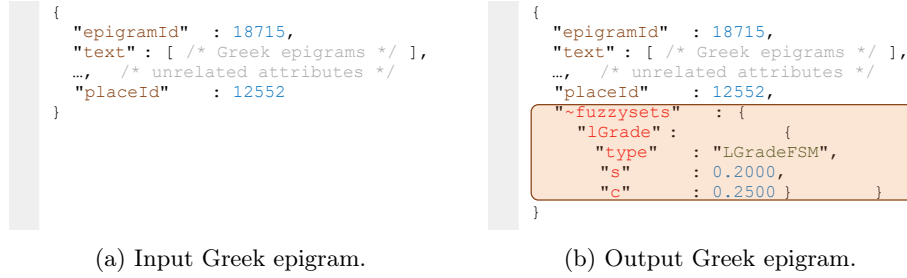


Fig. 1: Examples of input (left) and output (right) Greek epigram documents.

A primary veracity issue in the DBBE is the prevalence of missing or coarse geographical data. Within a representative set of 494 epigrams, 85.8% (423) lack geographical information entirely, while 5.4% are described only at coarse levels, with a granularity of **country**, **part**, or **region**. This sparsity creates significant challenges for conventional querying. For example, when searching for epigrams written at a specific city, such as *Messina* (in *Sicily*, *Southern Italy*), conventional techniques retrieve only documents either explicitly tagged with that city or its finer-grained sub-locations. Consequently, epigrams annotated at coarser levels (such as the surrounding region *Sicily* or country *Italy*) are omitted, despite representing partially relevant geographical information that could be valuable for researchers. Because veracity is a multi-faceted concept, it cannot be interpreted in a single way. In this work only the concept of missing or partial information is considered.

Recent efforts have attempted to address sparse metadata via similarity calculation and clustering [7]. However, these methods are too general and remain ill-suited to the specific problem of geographical and hierarchical querying addressed in this paper.

2.3 Soft Querying with the J-CO Framework

In the last decade, the authors from the University of Bergamo (Italy) have worked on the development of the *J-CO* Framework. It encompasses tools for querying collections of *JSON* documents in the context of Big Data. The core innovation provided by the *J-CO* Framework is its query language, named *J-CO-QL*⁺. Among all its various features, it is specifically designed to provide analysts with advanced capabilities for “Soft Querying” *JSON* datasets. Hereafter, a brief introduction to the main features provided by *J-CO-QL*⁺ mainly related to the presented research follows.

- *J-CO-QL*⁺ allows users to write scripts that acquire *JSON* datasets from Web sources and *JSON* stores, transform and integrate them, perform soft queries and save results again into *JSON* stores.

Listing 1 *J-CO-QL⁺* Script: Defining fuzzy set model LGradeFSM.

```

1. CREATE FUZZY SET MODEL LGradeFSM
   DEGREES s, c                                // s:satisfaction, c:confidence
   OPERATOR NOT                                     // negation:      z = NOT x
     EVALUATE s AS 1-x.s
     EVALUATE c AS x.c
   OPERATOR AND                                     // conjunction: z = x AND y
     EVALUATE s AS MIN(x.s, y.s)
     EVALUATE c AS IF((x.s = 0 AND y.s = 0), MAX(x.c, y.c),
                       IF((x.s = 0), x.c,
                           IF((y.s = 0), y.c, MIN(x.c, y.c))))
   OPERATOR OR                                     // disjunction: z = x OR y
     EVALUATE s AS MAX(x.s, y.s)
     EVALUATE c AS IF((x.s = 1 AND y.s = 1), MAX(x.c, y.c),
                       IF((x.s = 1), x.c,
                           IF((y.s = 1), y.c, MIN(x.c, y.c)))));

```

- *J-CO-QL⁺* addresses a large variety of issues related to processing *JSON* documents. Declarative statements for manipulating and merging documents are provided, even concerning geo-tagging of *JSON* documents (see [13]).
- *J-CO-QL⁺* natively supports fuzzy sets [8]: *JSON* documents are “fuzzified”, in such a way the membership degrees to possibly multiple fuzzy sets are added to their representation. Soft conditions can be evaluated over those membership degrees for soft querying the *JSON* documents. Since membership degrees represent the degree with which a *JSON* document satisfies soft queries, they can be further used to rank query results. Fuzzy sets can be evaluated on geo-tagging, through fuzzy spatial relationships (see [9]).
- The latest version of *J-CO-QL⁺* allows for defining “multi-grade fuzzy-set models”, i.e., models that explicitly manage more than one degree, such as Intuitionistic Fuzzy Sets [2] and Pythagorean Fuzzy Sets [15]. The language does not provide a predefined pool of multi-grade models; in contrast, it allows analysts to define their own multi-grade model, as well as the operators to combine them in complex queries [10].

L-grade fuzzy sets are a particular case of multi-grade fuzzy sets; therefore, it was straightforward to exploit *J-CO-QL⁺* to practically experiment their adoption on practical cases.

Listing 1 reports the *J-CO-QL⁺* instruction that defines this model.

- The statement **CREATE FUZZY SET MODEL** defines a new fuzzy-set model. Specifically, Line 1 in Listing 1 defines the model named **LGradeFSM**, which is a concrete instantiation of the L-grade model.
- The clause **DEGREES** declares the quantitative components of the L-grade model. Two degrees are defined: **s** (satisfaction) and **c** (confidence).
- The clause **OPERATOR** allows for defining logical operators on instances of the fuzzy-set model under definition. Unary operators (e.g., **NOT**) take a single operand **x**, while binary operators (e.g., **AND**, **OR**) take two operands **x** and **y**. Within each clause **OPERATOR**, the sub-clause **EVALUATE** defines how each degree of the resulting instance is computed. The syntax **EVALUATE <degree> AS <expression>** assigns an expression to each degree, thereby directly implementing the L-grade rules for negation, conjunction, and disjunction (the

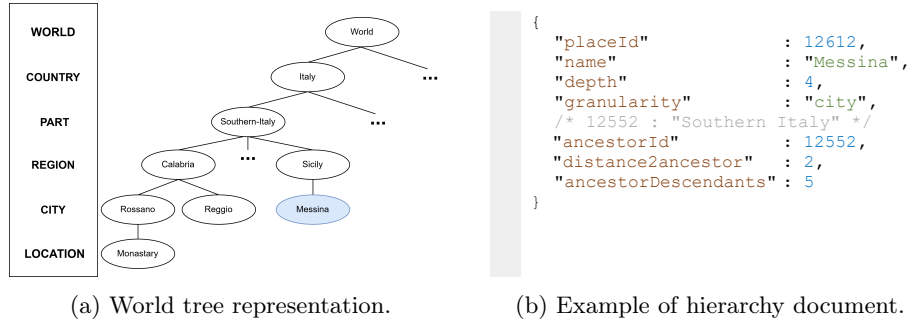


Fig. 2: Geographical hierarchy.

three parameters of the conditional function IF are, respectively, the condition, the true case, and the false case).

As an example, consider the sample epigram document reported in Fig. 1a. After its L-grade is evaluated (see Sect. 3.2), a special field named `~fuzzysets` is added to the document, as shown in Fig. 1b. Specifically, the complex field `lGrade` denotes this L-grade, which is a fuzzy set of type `LGradeFSM` (see Listing 1), with satisfaction 0.2000 (field `s`) and confidence 0.2500 (field `c`).

3 Flexible Querying based on a Geographical Hierarchy

The contribution of the paper is hereafter presented, i.e., a technique for flexibly querying based on a geographical hierarchy (Sect. 3.1) and its practical implementation through the *J-CO* Framework (Sect. 3.2). The experimental evaluation follows (Sect. 3.3).

3.1 An L-grade Model for Geographical Querying of the DBBE

To enable flexible querying of epigrams based on their location of origin, a model that is based on L-grades is proposed. While this model is specifically tailored to the DBBE geographical hierarchy, it remains easily adaptable to any hierarchical structure.

A fragment of the hierarchy is illustrated in Fig. 2a. Epigrams lacking a specific place are implicitly assumed to be associated with the root (i.e., `world`); the root serves as a catch-all element for all epigrams missing geographical information (this approach has been approved by historians).

When querying for epigrams associated with a specific geographical location q , each retrieved document (epigram) located at place p is assigned an L-grade (s, c) , determined by the hierarchical relationship between q and p in the geographical tree. The satisfaction degree is computed by Formula (7).

$$s = \begin{cases} 1 & \text{if } p = q \vee p \in \text{desc}(q) \\ \frac{1}{|\{d \in \text{desc}(p) : \text{depth}(d) = \text{depth}(q)\}|} & \text{if } p \in \text{anc}(q) \\ 0 & \text{otherwise} \end{cases} \quad (7)$$

The confidence is defined by Formula (8).

$$c = \begin{cases} \frac{1}{2^{\text{depth}(q) - \text{depth}(p)}} & \text{if } p \in \text{anc}(q) \\ 1 & \text{otherwise} \end{cases} \quad (8)$$

In Formulas (7) and (8), $\text{desc}(x)$ denotes the set of descendants of a location x , $\text{anc}(x)$ is the set of ancestors of x , and $\text{depth}(x)$ represents the depth (or granularity) of node x , with **world** situated at 0.

The satisfaction degree depends on the hierarchical relation between the query location q and the epigram location p : (1) If p is equal to or more specific than q (i.e., located at or below q in the hierarchy), the satisfaction is full ($s = 1$); (2) When p is a coarser ancestor of q , the satisfaction is partial and proportional to the number of descendant nodes of p at the same depth as q ; (3) If p and q lie on different branches of the hierarchy, there is no overlap and $s = 0$.

The confidence quantifies the reliability of the query result: (1) When p is an ancestor of q , the confidence strongly decreases with the growth of the distance between them in the tree, since a document associated with p could originate from multiple descendant locations of q ; (2) In all other cases, when either $p = q$, p is more specific than q , or p lies outside the path from q to the root node, the confidence is full ($c = 1$).

In summary, when p is contained within q , the query is fully satisfied with full confidence. When p contains q , the query is only partially satisfied and the confidence depends on the hierarchical distance between p and q . When p and q belong to distinct branches of the hierarchy, the query is not satisfied at all with full confidence.

This approach effectively bridges the gap between precisely localized epigrams and those with partial or totally missing geographic information, providing researchers with a nuanced view of the corpus that traditional querying cannot achieve.

3.2 L-grade Flexible Querying within the J-CO Framework

This section shows how the approach based on L-grades, presented in Sect. 3.1, can be practically applied through *J-CO-QL*⁺ on a *JSON* document store.

Reformulating the Query. In order to execute the query within *J-CO-QL*⁺, it is necessary to reformulate Formulas (7) and (8). The goal of this reformulation is to have one single mathematical expression for computing the satisfaction and one for computing the confidence. Formulas (7) and (8) can be rewritten as follows.

Listing 2 *J-CO-QL*⁺ Script: Defining fuzzy evaluator lgFuzzyfyFE.

```

2. CREATE LGradeFSM FUZZY EVALUATOR lgFuzzyfyFE
  PARAMETERS
    numerator          TYPE NUMERIC,      // parameter n in Formulas (9)
    ancestorDescendants TYPE NUMERIC,      // parameter d in Formulas (9)
    distance2ancestor  TYPE NUMERIC      // parameter e in Formulas (9)
  PRECONDITION (numerator = 0 OR numerator = 1) AND
    ancestorDescendants >= 1 AND
    distance2ancestor >= 0
  EVALUATE s AS numerator / ancestorDescendants
  EVALUATE c AS 1 / 2^distance2ancestor;

```

$$s = \frac{n}{d} \qquad c = \frac{1}{2^e} \quad (9)$$

In Formulas (9), n is the numerator, whose value can be either $n = 0$ or $n = 1$; d is the denominator, whose value is $d \geq 1$; e is the exponent for computing the confidence, whose value must be $e \geq 0$. Formulas (9) are equivalent to Formulas (7) and (8), assuming that n , h , and d take the values as reported hereafter.

- When q and p are not in the same hierarchical path, $n = 0$, otherwise $n = 1$; this ensures the proper computation of the satisfaction.
- When $p \in \text{anc}(q)$, d is the number of descendants of p at the same level of q ; in the other cases, $d = 1$ (first and third case of Formula (7)).
- When $p \in \text{anc}(q)$, $e = \text{depth}(q) - \text{depth}(p)$, is the distance in the hierarchy between q and p , otherwise, $e = 0$.

In this way, provided that n , d , and e are available for each document, it becomes easy to compute the corresponding L-grade.

Defining the *J-CO-QL*⁺ Evaluator. In order to fuzzify *JSON* documents on the basis of a given fuzzy-set model (see Listing 1), in *J-CO-QL*⁺ it is necessary to define a “fuzzy evaluator” [11], whose goal is to evaluate the extent with which the document belongs to a fuzzy set. Listing 2 reports the *J-CO-QL*⁺ definition of the fuzzy evaluator named lgFuzzyfyFE for the model LGradeFSM.

- The **PARAMETER** clause defines the parameters of the evaluator. There are 3 numerical values: in Formulas (9), **numerator** corresponds to n , **ancestorDescendants** corresponds to d , and **distance2ancestor** corresponds to e .
- The clause **PRECONDITION** specifies the constraints on the parameters, in order to ensure the correct evaluation of the L-grade model.
- The two clauses **EVALUATE** are responsible for actually evaluating the degrees **s** (satisfaction) and **c** (confidence), based on the actual values of the parameters. The reader can see that they correspond to Formulas (9).

This definition is preliminary to the actual execution of the query (Listing 3), in which the fuzzy evaluator will be used.

Listing 3 *J-CO-QL⁺* Script: Data analysis.

```

3. USE DB ipmuDb ON SERVER MongoDB 'http://127.0.0.1:27020';

4. JOIN OF COLLECTIONS Epigrams@ipmuDb AS P, PlaceHierarchy@ipmuDb AS H
CASES
  WHERE .E.placeId = 12612 OR
        (.E.placeId = .H.placeId AND .H.ancestorId = 12612)
  GENERATE
    ADD FIELDS {
      .numerator          : 1,
      .ancestorDescendants  : 1,
      .distance2ancestor   : 0
    }
  WHERE .E.placeId = .H.ancestorId AND .H.placeId = 12612
  GENERATE
    ADD FIELDS {
      .numerator          : 1,
      .ancestorDescendants  : .H.ancestorDescendants,
      .distance2ancestor   : .H.distance2ancestor
    }
  WHERE .E.placeId != .H.placeId AND
        .H.ancestorId = -1 AND .H.placeId = 12612
  GENERATE
    ADD FIELDS {
      .numerator          : 0,
      .ancestorDescendants  : 1,
      .distance2ancestor   : 0
    };

5. GROUP PARTITION WITH .E
  BY .E
  INTO .group
  GENERATE
    ADD FIELDS {
      .numerator          : MAX_ARRAY(.group, NUMERIC, .numerator),
      .ancestorDescendants  : MAX_ARRAY(.group, NUMERIC, .ancestorDescendants),
      .distance2ancestor   : MAX_ARRAY(.group, NUMERIC, .distance2ancestor) };

6. FILTER
CASES WHERE WITH .E
  GENERATE
    CHECK FOR LGradeFSM FUZZY SET lGrade
    USING lgFuzzyfyFE (.numerator, .ancestorDescendants, .distance2ancestor)
    BUILD { UNNEST .E };

7. SAVE AS LGradeEpigrams@ipmuDb;

```

Executing the Query. In order to execute the query, preprocessing was performed on the original *JSON* representation of the hierarchy depicted in Fig. 2a (i.e., one single *JSON* document). A dedicated *J-CO-QL⁺* script preprocessed the hierarchy in order to obtain one document for each ancestor-descendant pair in the hierarchy. Figure 2b reports a sample document: the field `placeId` identifies a place (node) in the hierarchy, whose level is denoted by the field `name`, and whose depth in the hierarchy is denoted by the field `depth` (the corresponding granularity is reported by the field `granularity`). The field `ancestorId` denotes an ancestor of `placeId`, such that the distance from the ancestor is denoted by the field `distance2ancestor`; finally, the field `ancestorDescendants` is the number of descendants of the ancestor at the same level of `placeId` (i.e., the number of descendants at the same level of `placeId`).

Suppose that the user looks for epigrams that were written in the city of *Messina (Italy)*, i.e., $q = \text{Messina}$. Listing 3 reports the *J-CO-QL⁺* script that actually performs the query. The reader can note that the datasets are stored within a database (named `ipmuDb`) managed by *MongoDB*; Line 3 connects the script to the database.

Line 4 in Listing 3 joins the collection `epigrams@ipmuDb` (aliased as `E`), which contains epigrams (see Fig. 1a), and the collection `PlacesHierarchy@ipmuDb` (aliased as `H`), which contains documents that describe ancestor-descendant pairs in the hierarchy (see Fig. 2b); both collections come from the database. A pair is represented by a *JSON* document that contains the fields `E` and `H` (which are the documents coming from the collections `E` and `H`, respectively); such documents are later denoted as *D*. The clause `CASE` evaluates three cascading conditions (after the keywords `WHERE`), which work on each single document *D*; when a given condition is met, a document is generated in the output collection by the corresponding sub-clause `GENERATE`; all of the three sub-clauses `GENERATE` generate documents with the same structure, i.e., they add three fields to the document *D* (keyword `ADD FIELDS`). Note that the wanted place *q* is reported with its identifier 12612 (i.e., *Messina*) within the script.

- The first `WHERE` condition checks for the case in which the epigram `E` is associated, through the document `H`, to a place *q* in the upper levels of the hierarchy, as well as the case in which the epigram is in the queried place *q*. As a result, *D* is extended with the fields `numerator` (set to 1), `ancestorDescendants` (set to 1), and `distance2ancestor` (set to 0).
- If the first condition `WHERE` is false, the second one is evaluated. It checks for cases in which an epigram `E` is paired with a document `H` where *q* is a descendant of the epigram place. Again, *D* is extended with the three fields `numerator` (set to 1), `ancestorDescendants` that reports the number of descendants of the epigram place (set to `H.ancestorDescendants`), and `distance2ancestor` (set to `H.distance2ancestor`).
- If the two previous conditions are false, the third condition `WHERE` is evaluated. It checks for the case in which the epigram `E` is paired with a document from `H` such that it associates the place *q* with the root of the hierarchy, and the epigram place is not *q*. So, *D* is extended with the fields `numerator` (set to 0), `ancestorDescendants` (set to 1), and `distance2ancestor` (set to 0).

As a result, for each epigram either one or two documents are generated, having the same field `E`. Therefore, the instruction on Line 5 groups all the documents having the same epigram `E`, so that one single document *G* is generated with the fields `E` and `group`, which contains all the grouped documents. The clause `GENERATE` extends *G* with the maximum value for the fields `numerator`, `ancestorDescendants`, and `distance2ancestor` that comes from the documents produced by Line 4 and grouped within the array `group`. The result is a collection that contains one single *JSON* document for each epigram, whose fields `numerator`, `ancestorDescendants`, and `distance2ancestor` are set with the proper values to compute the L-grade.

Line 6 is then able to generate the fuzzified epigram documents. Namely, the `CHECK FOR` clause evaluates the L-grade named `lGrade`, by evaluating the soft condition after the keyword `USING`. The soft condition calls the fuzzy evaluator `lgFuzzyfyFE` (see Listing 2), by passing the values of the fields `numerator`, `ancestorDescendants`, and `distance2ancestor` as actual parameters. The resulting documents are provided with the `~fuzzysets` field, with the evaluated

	c=0.0625	c=0.1250	c=0.2500	c=0.5000	c=1.0000
s=0.0000					62
s=0.0500	423				
s=0.1250		4			
s=0.2000			2		
s=1.0000				1	2

Fig. 3: Confusion matrix indicating the resulting L-grades when querying the database for epigrams written in the city *Messina*.

L-grade. Finally, the sub-clause **BUILD** creates the final structure of the output document(s), i.e., all and only the fields within the former field **E**, unnested at the root level (directive **UNNEST**). The resulting documents are like the sample document depicted in Fig. 1b, i.e., the epigram extended with the `~fuzzysets` field that reports the computed L-grade.

The resulting collection is saved into the database, as **LGradeEpigrams**, for later exploitation and analysis. As a typical activity, ranking results through an ordered list is a straightforward way to exploit results, which could rely on the ordering relations $\succ_{lex}^s$ and $\succ_{lex}^c$ presented in Formulas (6).

3.3 Experimental Evaluation

A basic experimental evaluation was performed to assess the suitability of the proposed technique. Using the script in Listing 3, the 494 documents were queried, looking for those associated with the city of *Messina* (located in *Sicily, Italy*), computing an L-grade for each epigram. The process yielded a small set of distinct satisfaction and confidence value pairs; the distribution of documents across these (s, c) pairs is illustrated in the confusion matrix in Fig. 3.

Of the three epigrams retrieved with full satisfaction ($s = 1$), two were located within *Messina* or its sub-regions; these represent exact matches and are considered fully reliable ($c = 1$). The third was associated with the broader region of *Sicily*. While it maintains full satisfaction ($s = 1$) because *Messina* is the only city in *Sicily* that has emerged from the collected epigrams, the confidence reduces to $c = 0.5$. It is thus a full match, albeit with moderate confidence.

Additionally, two epigrams with $s = 0.2$ and four with $s = 0.125$ correspond to locations identified only as *Southern Italy* and *Italy*, respectively. These satisfaction values are proportional to the number of cities (the queried granularity) within those regions, indicating a partial match. The increased distance from the target granularity results in lower veracity scores ($c = 0.25$ and $c = 0.125$), reflecting the diminished reliability of such broad geographical descriptions.

62 epigrams do not match the query at all ($s = 0$); since these are associated with known locations unrelated to *Messina*, they get full confidence ($c = 1$). The remaining 423 epigrams lack geographical descriptions and are assumed to have been written somewhere in the world. Because they could potentially have originated in *Messina*, they are assigned a very low satisfaction ($s = 0.05$) with very poor confidence ($c = 0.0625$).

While not an exhaustive validation, this evaluation provides a compelling initial indication of the technique’s capabilities. It demonstrates how the proposed approach addresses not only the retrieval of relevant data items but also quantifies their veracity relative to the user’s query.

4 Conclusions

This paper has proposed a novel technique for querying datasets containing geographical descriptions, specifically addressing scenarios where missing or imprecise information creates data-veracity concerns. Unlike traditional methods that omit partial matches, or standard flexible-querying techniques that overlook data reliability, this approach introduced a more rigorous framework. The central contribution is the utilization of the L-grade model, a multi-grade extension of classical fuzzy sets, which associates each query result with both a degree of satisfaction and a degree of confidence (veracity).

A further novelty of this work is the practical implementation of this technique within the *J-CO* Framework. It is the first framework to provide sophisticated soft-querying capabilities on top of diverse *JSON* document stores, the current standard for Big Data management. This implementation was tested on the documents stored in the Database of Byzantine Book Epigrams, demonstrating that *J-CO* can be used as a concrete and effective tool for handling the complex nature of data veracity through L-grades.

As future work, a more rigorous assessment will be performed by investigating a larger volume of more complex queries across various domains with deeper data hierarchies. Since veracity is not limited to missing or incomplete information, other sources of data veracity will be investigated. Additionally, the authors will explore the introduction of complex queries based on logical operators, and they will study various ranking approaches for query results to further assist users in effectively exploiting the results of these veracity-aware queries.

The *J-CO* Framework², the scripts and the datasets³ shown in this paper are available for free on a GitHub repository.

Acknowledgments. The research part on veracity handling has been funded by the Flemish Government (AI Research Program).

The research part on the *J-CO* Framework has been funded by the European Union - *NextGenerationEU*, in the framework of the *GRINS - Growing Resilient, INclusive and Sustainable project* (GRINS PE00000018 – CUP F83C22001720001). The views and opinions expressed are solely those of the authors and do not necessarily reflect those of the European Union, nor can the European Union be held responsible for them.

Disclosure of Interests. The authors have no competing interests to declare.

² The *J-CO* Framework: <https://github.com/JcoProjectTeam/JcoProjectPage>, accessed on 30/01/2026.

³ Data and scripts: <https://github.com/JcoProjectTeam/JcoProjectPage/blob/main/papers/pages/IPMU2026.md>, accessed on 30/01/2026.

References

1. Andreassen, T., Bordogna, G., De Tré, G., Kacprzyk, J., Larsen, H.L., Zadrozny, S.: The power and potentials of flexible query answering systems: A critical and comprehensive analysis. *Data & Knowledge Engineering* **149**, 102246 (2024). <https://doi.org/10.1016/j.datak.2023.102246>
2. Atanassov, K.T.: Intuitionistic fuzzy sets. *Fuzzy Sets and Systems* **20**(1), 87–96 (1986). [https://doi.org/10.1016/S0165-0114\(86\)80034-3](https://doi.org/10.1016/S0165-0114(86)80034-3)
3. Braga, D.D.S., Niemann, M., Hellingrath, B., Neto, F.B.D.L.: Survey on computational trust and reputation models. *ACM Computing Surveys (CSUR)* **51**(5), 1–40 (2018). <https://doi.org/10.1145/3236008>
4. Chen, J., Chen, Y., Du, X., Li, C., Lu, J., Zhao, S., Zhou, X.: Big data challenge: a data management perspective. *Frontiers of computer Science* **7**(2), 157–164 (2013). <https://doi.org/10.1007/s11704-013-3903-7>
5. De Tré, G., Peelman, M., Dujmović, J.: Logic reasoning under data veracity concerns. *International Journal of Approximate Reasoning* **161**, 108977 (2023). <https://doi.org/10.1016/j.ijar.2023.108977>
6. Deforche, M., De Tré, G.: Ranking l-grades for veracity handling in logic reasoning. In: *International Fuzzy Systems Association World Congress & North American Fuzzy Information Processing Society Annual Meeting, IFSA-NAFIPS, 2025, Proceedings* (in press). Springer Cham (2026)
7. Deforche, M., De Vos, I., De Tré, G.: A novel approach to context-aware and responsible short text clustering. In: *Flexible Query Answering Systems : 16th International Conference, FQAS, 2025, Proceedings*. vol. 16119, pp. 19–30. Springer Cham (2026). https://doi.org/10.1007/978-3-032-05607-8_4
8. Fosci, P., Psaila, G.: J-co, a framework for fuzzy querying collections of json documents. In: *Flexible Query Answering Systems: 14th International Conference, FQAS, 2021, Proceedings*. pp. 142–153. Springer International Publishing (2021). https://doi.org/10.1007/978-3-030-86967-0_11
9. Fosci, P., Psaila, G.: Soft spatial querying on json data sets. In: *European Conference on Advances in Databases and Information Systems*. pp. 223–237. Springer Cham (2022). https://doi.org/10.1007/978-3-031-15740-0_17
10. Fosci, P., Psaila, G.: A unified view of multi-grade fuzzy-set models in j-co-ql+. *Neurocomputing* **565**, 126968 (2024). <https://doi.org/10.1016/j.neucom.2023.126968>
11. Fosci, P., Psaila, G.: Evolving j-co-ql+ with fuzzy evaluators for flexible querying of json data sets. *Neurocomputing* **633**, 129621 (2025). <https://doi.org/10.1016/j.neucom.2025.129621>
12. Meier, A., Kaufmann, M.: *SQL & NoSQL databases*. Springer (2019). <https://doi.org/10.1007/978-3-658-24549-8>
13. Psaila, G., Fosci, P.: J-co: A platform-independent framework for managing geo-referenced json data sets. *Electronics* **10**(5), 621 (2021). <https://doi.org/10.3390/electronics10050621>
14. Ricceri, R., et al.: The Database of Byzantine Book Epigrams Project: Principles, Challenges, Opportunities. *Journal of Data Mining & Digital Humanities* (2023). <https://doi.org/10.46298/jdmdh.10244>
15. Yager, R.R.: Pythagorean membership grades in multicriteria decision making. *IEEE Transactions on fuzzy systems* **22**(4), 958–965 (2013). <https://doi.org/10.1109/TFUZZ.2013.2278989>
16. Zadeh, L.A.: A note on z-numbers. *Information sciences* **181**(14), 2923–2932 (2011). <https://doi.org/10.1016/j.ins.2011.02.022>